

Revision of batch updates

17 August 2026 09:04

Batch update is a JDBC feature used to execute multiple **insert, update or delete** operations. Together instead of sending each SQL statement separately to the Oracle database.

This is especially useful when processing a large number of records, such as:

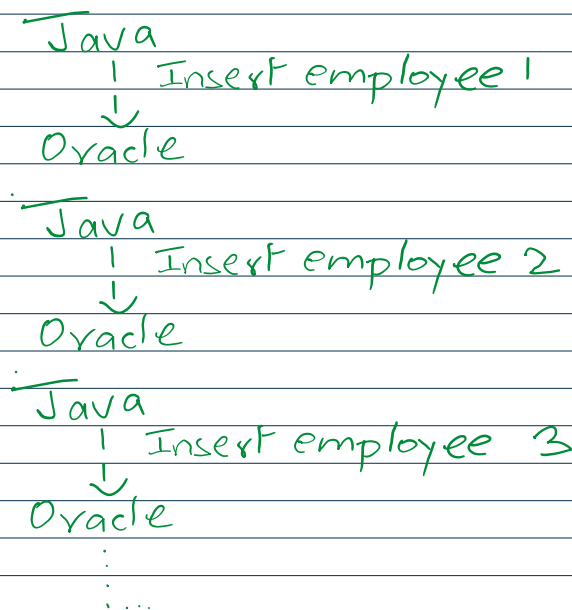
- Adding 1000 employees.
- Updating salaries for many employees.
- Importing student records.
- Processing payroll.
- Bulk order insertion.
- Update inventory.

Oracle's current JDBC documentation recommends the standard JDBC batching model using `addBatch()` and `executeBatch()` rather than older Oracle-specific batch APIs.

What is batch update?

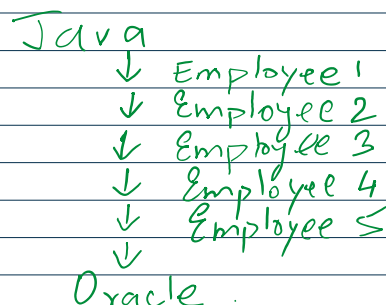
Suppose we want to insert five employees.

Without batch processing:



There are multiple communication round trips between Oracle and Java.

With batch processing:

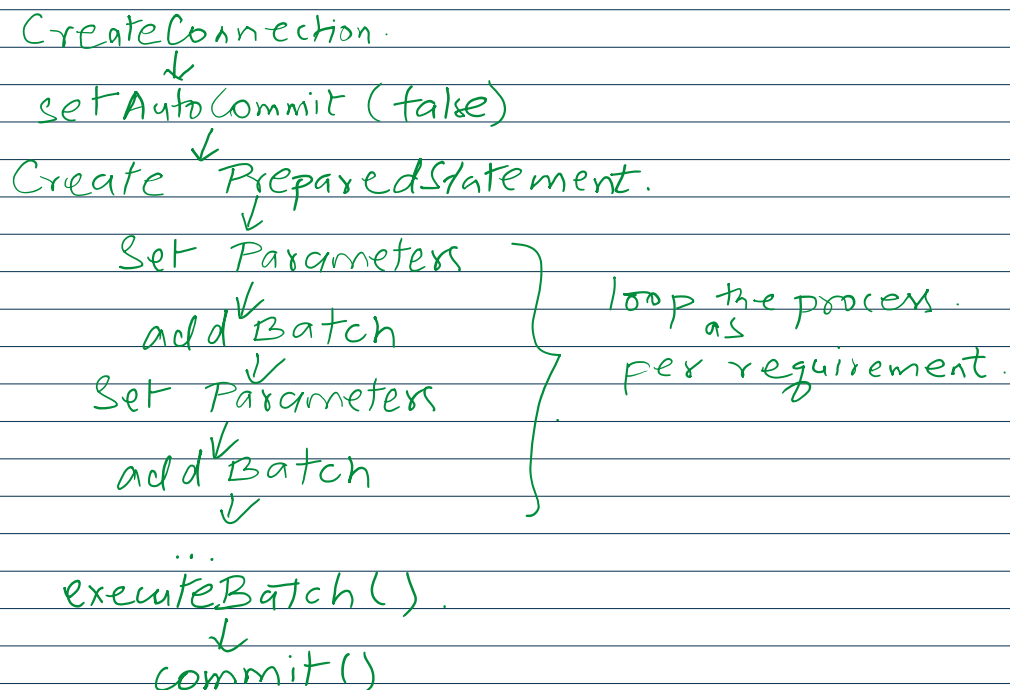


The operations are collected into a batch and submitted using `executeBatch()`.

This can reduce database round trips and improve performance, particularly for repeated operations using a `PreparedStatement`.

Basic Batch Processing Flow

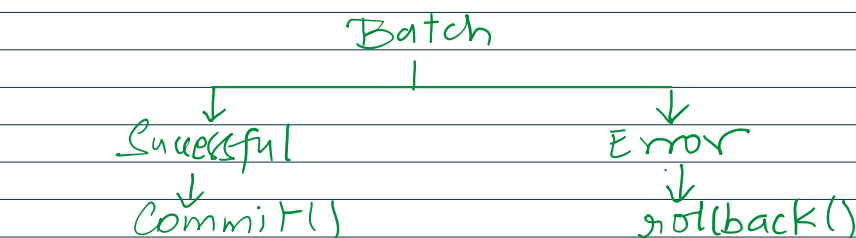
The standard JDBC, batch process is:



The important methods are:

1. `addBatch()`
2. `executeBatch()`
3. `clearBatch()`

Oracle recommends disabling AUTO - commit for batch processing so that the application can properly commit or rollback changes when an error occurs.



try {

```
con.setAutoCommit(false);  
// add batch operation.
```

```
int rows = stmt.executeUpdate();
```

```

// add batch operation.
int[] result = pst.executeBatch();
con.commit();
System.out.print("Transaction completed");
} catch (SQLException se) {
    try {
        con.rollback();
        System.out.print("In Transaction rollback.");
    } catch (SQLException se) {
        se.printStackTrace();
    }
    se.printStackTrace();
}
}

```

This is much more safer for real application.

Batch size

Suppose you have. 100,000 employees.

You don't necessarily want to pull all 100,000 operations into one batch. Instead you can process them in a smaller groups.

For example:

Records

```

|
|-- Batch-1 --> 1 to 500
|-- Batch-2 --> 501 to 1000
|-- Batch-3 --> 1001 to 1500
|...

```

```

int batchSize = 500;
int i;
for(i = 1; i <= 10000; i++){
    //set parameters
    pst.addBatch();
    if(i % batchSize == 0){
        pst.executeBatch();
    }
}

```

remainingRecords = i % 500;

But remember to execute any remaining records after the loop.

```

if(remainingRecords > 0){
    pst.executeBatch();
}

```

Oracle's documentation also notes that extremely large batches can create memory/performance issues, so practical batch sizes should be chosen according to the applications workload.

Why PreparedStatement is Better?

```
String sql = "INSERT INTO employee " +  
            "VALUES (?, ?, ?, ?) ";
```

```
pst.setString(1, "E001");
```

```
...  
pst.addBatch();
```

```
pst.setString(1, "E002");
```

```
...  
pst.addBatch();
```

```
pst.setString(1, "E003");
```

```
...  
pst.addBatch();
```

```
pst.executeBatch();
```

This provides:

- Cleaner code
- Parameter binding
- Better protection against SQL injection.
- Efficient repeated execution
- Natural integration with JDBC batching.

`clearBatch()`: If you want to remove operations that have been added to the batch but not yet executed.

```
pst.clearBatch(); //removes pending operations
```

After `clearBatch()` the pending batch is empty.

The standard JDBC, API defines `clearBatch()` for clearing the current batch.

BatchUpdateException: When something goes wrong while executing a batch JDBC can throw `BatchUpdateException`. It is present in `java.sql` package.

```
try{  
    //batch statements  
}  
catch(BatchUpdateException e){  
    int[] counts = e.getUpdateCount();  
    for(int count : counts){  
        System.out.print("Update count: " + count);  
    }  
    con.rollback();  
}
```

Oracle/JDBC documents batch update exception as the exception used when a batch execution encounters an error and its update - count information can help determine what was processed.

Complete Transactional Batch Example

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
public class TransactionalBatchDemo {
    public static void main(String[] args) {
        Connection con = ConnectionObject.getConnection();
        String sql = "INSERT INTO employee " +
            "(emp_id, name, dept_id, basic_salary)" +
            "VALUES(?,?,?,?)";
        try (PreparedStatement pst = con.prepareStatement(sql)) {
            try {
                con.setAutoCommit(false);
                pst.setString(1, "E019");
                pst.setString(2, "Frank");
                pst.setString(3, "D003");
                pst.setInt(4, 67800);
                pst.addBatch();
                pst.setString(1, "E020");
                pst.setString(2, "Davis");
                pst.setString(3, "D004");
                pst.setInt(4, 62000);
                pst.addBatch();
                pst.setString(1, "E021");
                pst.setString(2, "Anthony");
                pst.setString(3, "IT");
                pst.setInt(4, 67900);
                pst.addBatch();
                pst.setString(1, "E022");
                pst.setString(2, "Issac");
                pst.setString(3, "Research");
                pst.setInt(4, 167900);
                pst.addBatch();
                int[] result = pst.executeBatch();
                for (int r : result) {
                    System.out.println(r);
                }
                con.commit();
                System.out.println("Batch committed Successfully.");
            } catch (SQLException e) {
                con.rollback();
                System.out.println("Batch failed. Transaction rolled back.");
                throw e; //Chaining of catch exception
            }
        } catch (SQLException e) {
            // e.printStackTrace();
            System.out.println("Check the batch query.");
        }
    }
}
```

Output:

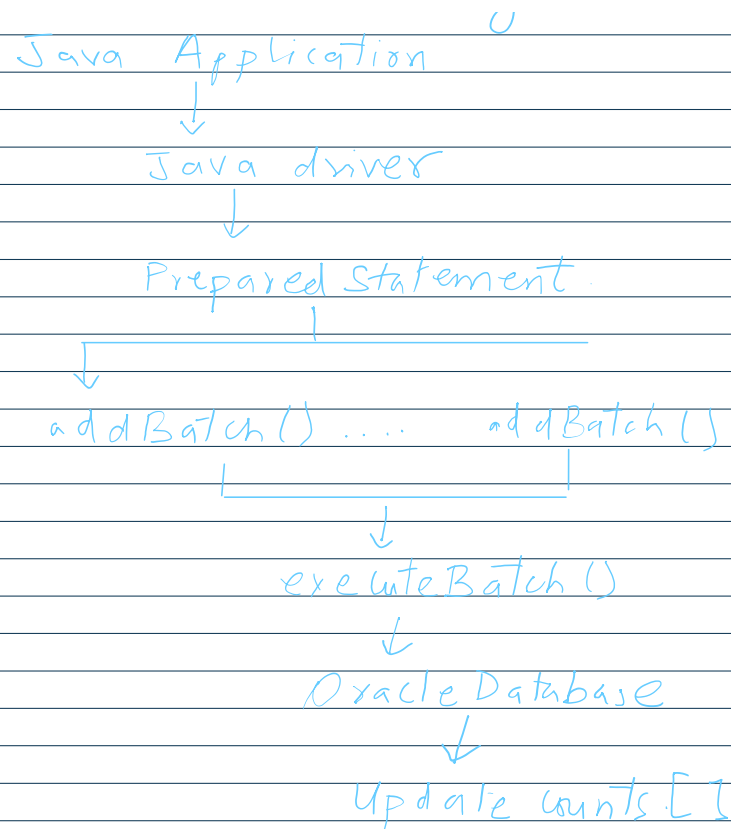
Batch failed. Transaction rolled back.

Check the batch query.

Large-Data Batch Example

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;
public class LargeBatchInsert {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Connection con = ConnectionObject.getConnection();
        String sql = "INSERT INTO employee " +
            "(emp_id, name, dept_id, basic_salary)" +
            "VALUES(?,?,?,?)";
        int batchSize = 100;
        try (PreparedStatement pst = con.prepareStatement(sql)) {
            try {
                con.setAutoCommit(false);
                for (int i = 1; i <= 1000; i++) {
                    System.out.println("Enter employee id: ");
                    pst.setString(1, sc.next());
                    System.out.println("Enter employee name: ");
                    pst.setString(2, sc.nextLine());
                    System.out.println("Enter department id: ");
                    pst.setString(3, sc.next());
                    System.out.println("Enter basic salary: ");
                    pst.setInt(4, sc.nextInt());
                    pst.addBatch();
                    if(i % batchSize == 0){
                        pst.executeBatch();
                        pst.clearBatch();
                        System.out.println(i + " record processed.");
                    }
                }
                con.commit();
                System.out.println("Batch committed Successfully.");
            } catch (SQLException e) {
                con.rollback();
                System.out.println("Batch failed. Transaction rolled back.");
                throw e; // Chaining of catch exception
            }
        } catch (SQLException e) {
            // e.printStackTrace();
            System.out.println("Check the batch query.");
        }
    }
}
```

Batch Processing Architecture
Java Application
1



H.W

Importing Employee Data stored in csv file

emp_id	name	dep_id	basic_salary
1000	XX	XXX	XXXX.XX
:			

employees.csv

csv File



BufferedReader



Read each line



split csv fields



setAutoCommit(false)

Prepared statement



setXXX(index, fields)

:

↓
addBatch()



executeBatch()

↳ commit()

Oracle database



return [] counts

